MMF2030H — Machine Learning

Assignment 2

DUE DATE	November 10, 2026, 11:59 PM (one week after Lecture 5, one week before the final exam)
WEIGHT	20% of final grade
COVERS	Lectures 3–4 — trees & gradient boosting (XGBoost specifically), feature importance & SHAP, backpropagation & neural network architecture/regularization, unsupervised learning & HRP
FORMAT	Mostly applied/coding — this is where the coding skills from Assignment 1's theory are put into practice, including a small neural network and an XGBoost model.
SUBMISSION	PDF write-up + code (.ipynb or .py) via course portal; individual work

Instructions

- This assignment is completed individually. You may discuss concepts with classmates, but all derivations, code, and write-up must be your own.
- Part A (Theory) is two short warm-ups — a page of work each is normal. Part B (Applied) is the bulk of the assignment: runnable code plus a short written interpretation of your results; code without interpretation will not receive full credit.
- Submit a single PDF containing your Part A answers and Part B write-up (including all plots), and a separate notebook or script for Part B. State any assumptions you make.
- The provided credit dataset (mmf2030h_a2_credit_data.csv) marks each row's role in a split column. Train your B1/B2 models only on split == "train" rows, and report all performance metrics (AUC, etc.) on split == "test" rows only — do not re-shuffle or re-split the data, since the reference solution and grading tolerances assume this exact split. If you try the optional fine-tuning practice, the same discipline applies to the provided mmf2030h_a2_finbert_headlines.csv and its own split column — see README_A2_datasets.md for details on all three datasets.

Part A — Theory Warm-Ups (6 marks)

A1. Information gain by hand (3 marks). You are given a parent node with 20 loans, 5 of which default. A candidate split sends 14 loans to child L (2 defaults) and 6 loans to child R (3 defaults). Compute the entropy of the parent node, the entropy of each child, the weighted child entropy, and the resulting information gain. Show all intermediate values.

A2. Backpropagation by hand (3 marks). Consider a network with one hidden layer: input $x \in \mathbb{R}$, hidden unit $h = \sigma(w_1 x + b_1)$ with sigmoid activation σ, output $\hat{y} = w_2 h + b_2$, and squared-error loss $L = \frac{1}{2}(y - \hat{y})^2$. Derive $\partial L/\partial w_1$ by the chain rule, writing out each factor explicitly (the Lecture 4 backprop DAG slide walks through exactly this structure). Using $x = 1$, $w_1 = 0.5$, $b_1 = 0$, $w_2 = 1.0$, $b_2 = 0$, and $y = 1$, compute the numerical value of $\partial L/\partial w_1$.

Part B — Applied / Coding (14 marks)

- You will fit and interpret an XGBoost model, train and evaluate a neural network via architecture comparison and regularization, and compare a correlation-aware portfolio construction method to naive mean-variance. Three datasets built for this assignment (fixed, provided on the course portal) are specified below, so everyone works from identical data with a pre-assigned train/test split — no need to source or split anything yourself.

B1. XGBoost + SHAP (5 marks)

i. Use the provided dataset mmf2030h_a2_credit_data.csv (8,000 borrowers, 10 features including revolving credit utilization, debt ratio, monthly income, and number of past-due incidents, predicting serious_dlqin_2yrs, a binary default flag; ~10% base rate). Use the split column exactly as given — do not re-shuffle or re-split — so every submission trains and evaluates on identical rows. Fit an XGBoost classifier on the train rows, report AUC on the test rows, and briefly justify one hyperparameter choice (e.g., max_depth, min_child_weight, or the learning rate) in terms of the second-order gain formula from lecture.

ii. Compute both the model's default (MDI-style) feature importance and SHAP values (the shap package, or XGBoost's built-in SHAP support). Plot both side by side. Retain the per-instance (per-borrower) SHAP values you compute here, not just the summary plot — handy to have if you try the optional interpretability practice later.

iii. Discuss any features where MDI and SHAP importance disagree, and explain the disagreement using what you know from Lecture 3's treatment of MDI vs. MDA vs. SHAP.

B2. Neural network architecture & regularization (5 marks)

Complete the task below (i–iii). Grading is based on your PDF write-up — make sure all plots, reported numbers, and discussion are in it; the notebook/script is kept on file for spot-checks of methodology, not graded line by line.

i. Using the same dataset (or target) as B1, build at least two feedforward neural network architectures that differ in depth and/or width (e.g., a shallow-wide network vs. a deep-narrow network). Train both with backpropagation (any standard framework — PyTorch, TensorFlow/Keras, or your own NumPy implementation extending A2 is also acceptable).

ii. Apply one regularization technique from Lecture 4 (dropout, weight decay/L2, or early stopping) to your better-performing architecture, and show a training/validation loss curve with and without it.

iii. Report out-of-sample performance (the same metric as B1) for both architectures, with and without regularization, and compare all of these results against your B1 XGBoost model on the same held-out set. In two or three sentences, discuss which model you would deploy and why — consider not just accuracy but training cost, interpretability, and overfitting risk.

Optional — More Practice (not required, not graded)

If you want more hands-on practice beyond the required task above, you're welcome to try one of these on your own time. They are not submitted and not graded — purely for your own learning.

Fine-tuning with Hugging Face (modern NLP): using the provided mmf2030h_a2_finbert_headlines.csv, load a pretrained model (e.g. ProsusAI/finbert) and apply LoRA via the peft library, following the Lecture 4 slides on efficient fine-tuning. Evaluate zero-shot performance, then fine-tune and re-evaluate on the same test split.

Interpretability deep-dive: using the B1 credit data, train one feedforward neural network with one regularization technique from Lecture 4, then compute saliency maps and Integrated Gradients attributions for a handful of test-set borrowers and compare them to those borrowers' B1 SHAP values.

B3. Hierarchical Risk Parity vs. naive Markowitz (4 marks)

i. Use the provided dataset mmf2030h_a2_prices.csv — daily close prices, Jan 2020–Oct 2024 (1,250 trading days), for 8 tickers in two groups: TECH_A–D (expected to be internally correlated, higher-volatility) and STPL_A–D (expected to be internally correlated, lower-volatility), with low correlation across the two groups. Use all 8 tickers and the full date range — compute daily returns and build the empirical covariance matrix from those returns.

ii. Implement HRP (quasi-diagonalization, recursive bisection) and compute portfolio weights; compute weights for a naive mean-variance (Markowitz) portfolio on the same covariance matrix.

iii. Compare the two weight vectors and their concentration (e.g., effective number of assets, or weight variance). Briefly discuss why HRP tends to produce more diversified weights when the covariance matrix is ill-conditioned or cluster-correlated.

Deliverables Checklist

- PDF with Part A derivations (A1–A2) and Part B write-up, including all requested plots and discussion.
- Notebook or .py script for Part B, runnable top to bottom, with your simulation seed (if applicable) fixed for reproducibility.

Assignment 2 - Jaime Cozzo Galdó

Part A

A1) For a binary classification problem, the entropy of a node is given by:

$$H(p) = -p\log_2(p) - (1-p)\log_2(1-p)$$

→ Parent node: The parent contains 20 loans, of which 5 default.

$$p = \frac{5}{20} = 0,25$$

$$H(\text{parent}) = -0,25\log_2(0,25) - 0,75\log_2(0,75) = 0,8113 \text{ bits}$$

→ Child L: Contains 14 loans, of which 2 default.

$$p = \frac{2}{14} = 0,1429$$

$$H(L) = -0,1429\log_2(0,1429) - 0,8571\log_2(0,8571) = 0,5917 \text{ bits}$$

→ Child R: Contains 6 loans, of which 3 default.

$$p = \frac{3}{6} = 0.5$$

$$H(R) = -0,5\log_2(0,5) - 0,5\log_2(0,5) = 1 \text{ bit}$$

The weighted child entropy is $H(\text{weighted}) = \frac{14}{20}0,5917 + \frac{6}{20}\cdot 1 = 0,7142$

So the information gain is $\boxed{IG = H(\text{parent}) - H(\text{weighted}) = 0,0971 \text{ bits}}$

⇒ The split reduces the weighted average entropy from 0.8113 to 0,7142 bits, resulting in an information gain of approximately 0,0971 bits. Therefore, the split reduces uncertainty about loan default, although the improvement is relatively small.

A2| We are given a neural network with one hidden unit:

$$h = \sigma(w_1 x + b_1); \quad \hat{y} = w_2 h + b_2; \quad L = \frac{1}{2}(y - \hat{y})^2$$

1. Derivation using chain rule

Let $z_1 = w_1 x + b_1$. Applying the chain rule:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} \cdot \frac{\partial h}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Computing each factor:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

$$\frac{\partial \hat{y}}{\partial h} = w_2$$

$$\frac{\partial h}{\partial z_1} = h(1-h)$$

$h(z) = \sigma(z_1)$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \left(\frac{e^{-z}}{1+2e^{-z}+e^{-2z}} \cdot \frac{e^{-z}}{e^{-z}}\right) = \frac{e^{-z}}{1+e^{-z}} \cdot \frac{1}{1+e^{-z}} =$$

$$= \frac{(1+e^{-z})-1}{1+e^{-z}} \cdot \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right) = \sigma(z)(1-\sigma(z))$$

$$\frac{\partial z_1}{\partial w_1} = x$$

$$\Rightarrow \frac{\partial L}{\partial w_1} = (\hat{y} - y) w_2 h(1-h) x$$

2. Numerical Calculation

Given $x = 1$, $w_1 = 0.5$, $b_1 = 0$, $w_2 = 1$, $b_2 = 0$ and $y = 1$

$z_1 = 0.5(1) + 0 = 0.5$

$h = \sigma(0.5) \approx 0.62459$

$\hat{y} = \sigma(0.533459) + 0 = 0.533459$. Substituting into the gradient:

$$\Rightarrow \frac{\partial L}{\partial w_4} = -0.088723$$

The negative gradient indicates that increasing w_4 locally decreases the loss. Therefore, a gradient descent update would increase w_4, moving the prediction closer to the target value of 1

Part B

B1| i. XGBoost model and performance

I trained an XGBoost classifier using the provided dataset to predict serious_dlquin_2yrs. The dataset contains 8000 borrowers and 10 features.

The model was trained with 250 trees, max_depth = 3, learning_rate = 0.05, min_child_weight = 3 and reg_lambda = 1.

The resulting test AUC was 0.8291. This means that the model is reasonably good at distinguishing borrowers who default from those who do not. An AUC of 0.8291 indicates that a randomly selected defaulter will receive a higher predicted risk score than a randomly selected non defaulter approximately 82.9% of the time.

I chose min_child_weight = 3 to reduce overfitting. In XGBoost's second order gain formula:

$$Gain = \frac{1}{2}\left[G_L^2 \cdot \frac{1}{H_L + \lambda} + G_R^2 \cdot \frac{1}{H_R + \lambda} - \frac{G^2}{H + \lambda}\right] - \gamma$$

this parameter requires a minimum sum of Hessians in each child node. This prevents splits that rely on too little information and helps control model complexity.

ii. I calculated XGBoost's gain based feature importance and Tree Shap values.

Gain measures how much a feature improves the model's objective when used for splits during training. SHAP measures how much each feature contributes to individual model predictions.

I calculated the mean absolute SHAP values across the test set and normalized both importance measures to percentages for comparison

See MDI vs SHAP comparison plot in Appendix. - Figure 1

The most important feature according to both methods is revolving utilization, with 32,85% of total gain importance and 38,4% of total SHAP importance.

The number of payments 60-89 days past is also important, receiving 15,5% according to gain and 14,44% according to SHAP

iii. Although the two methods agree on the most important features, there are some differences.

For example, debt_ratio has a gain importance of 10,49% compared with 14,38% for SHAP.

In contrast, num_dependents receives 4,45% according to gain but only 0,92% according to SHAP.

These differences occur because the methods measure importance differently. Gain focuses improvements from training splits, while SHAP measures contributions to the model's predictions.

Also gain-based importance can favor continuous features with many possible split points and can distribute importance unevenly between correlated variables. SHAP provides an additional perspective on how features influence individual predictions, although neither method establishes causality.

B2| i Using the credit dataset from B1, I trained two feedforward neural networks with different architectures.

- Shallow-wide: One hidden layer with 64 neurons.
- Deep-narrow: Three hidden layers with 32, 16 and 8 neurons

Both networks use ReLU activations, the Adam optimizer, binary cross-entropy loss and 100 training epochs.

ii I applied L2 regularization (weight decay = 0,001) to control overfitting by discouraging excessively large network weights.

See loss curves plot in Appendix - Figure 2

With L2 regularization, the gap between training and validation losses is smaller and the final validation loss is slightly lower.

iii All AUC scores below were computed on the same held-out test set

Model	Regularization	Test AUC
Shallow-wide	None	0,8327
Shallow-wide	L2	0,8342
Deep-narrow	None	0,8057
Deep-narrow	L2	0,8264
XGBoost	—	0,8291

See AUC comparison plot - Figure 3 in Appendix

I would initially deploy XGBoost because its test AUC (0,8291) is close to the best NN's result (0,8342) while it offers strong performance on tabular data, convenient SHAP interpretability and relatively straightforward training.

B3| i Using the provided dataset, I computed daily simple returns for all eight assets, I then calculated the empirical covariance and correlation matrices of the daily returns

See correlation heatmap - Figure 4 in Appendix

The correlation matrix reveals two distinct clusters: TECH_A-D and STPL_A-D, with relatively high correlations within each group and lower correlations between groups.

ii I implemented Hierarchical Risk Parity (HRP) using correlation based hierarchical clustering, quasi diagonalization and recursive bisection. Cluster allocations were determined using their estimated variances.

See hierarchical clustering dendogram plot - Figure 5 in Appendix

For comparison, I constructed a long only Markowitz mean-variance portfolio using sample mean returns and the same empirical covariance estimates, annualized consistently.

iii See portfolio weights comparison chart - Figure 6 in Appendix

HRP allocates positive weights to all eight assets, while Markowitz concentrates nearly 48,17% of the portfolio in STPLC and assigns zero weight to four assets.

To quantify concentration, I computed the effective number of assets:

$$N_{eff} = \frac{1}{\sum w_i^2} \Rightarrow \begin{matrix} HRP = 5,57 \\ Markowitz = 3,06 \end{matrix}$$

HRP produces a more diversified distribution of portfolio weights in this sample.

HRP can be more robust when assets are highly correlated because it uses hierarchical clustering and recursive risk allocation rather than relying on the direct inversion of the covariance matrix.

In contrast, mean-variance optimization can produce concentrated and estimation-sensitive weights, particularly when covariance estimates are unstable.

CamScanner

Appendix

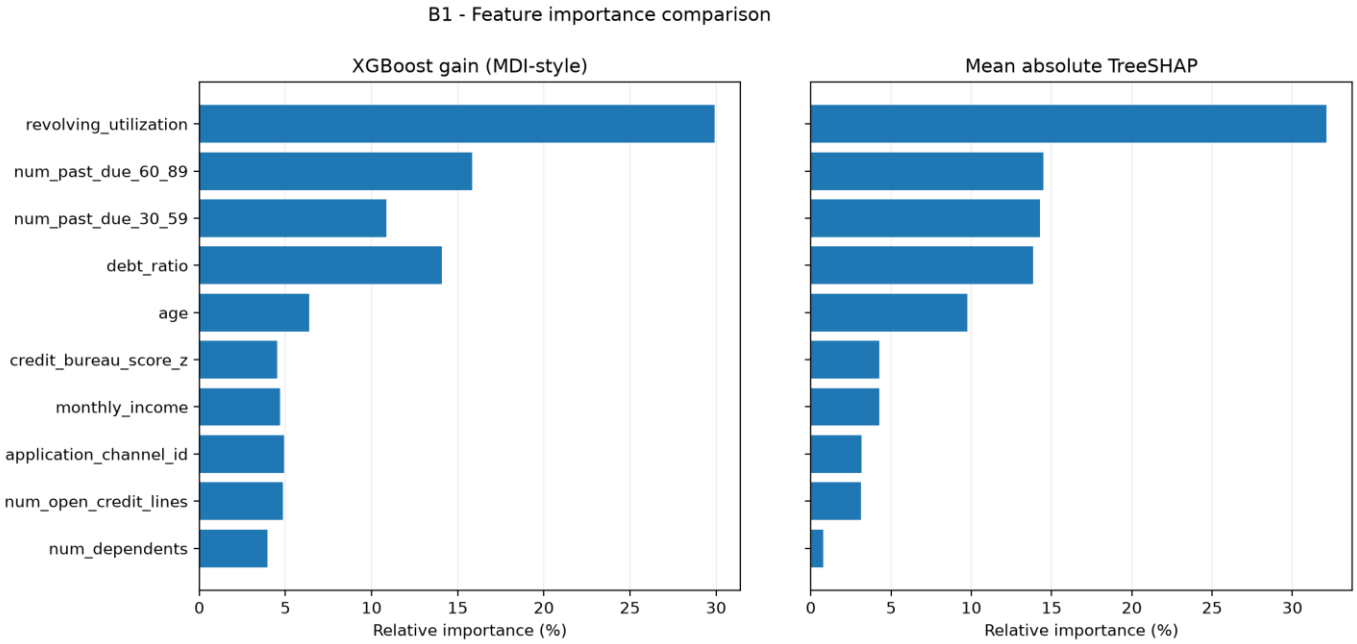


Figure 1- MDI vs SHAP comparison

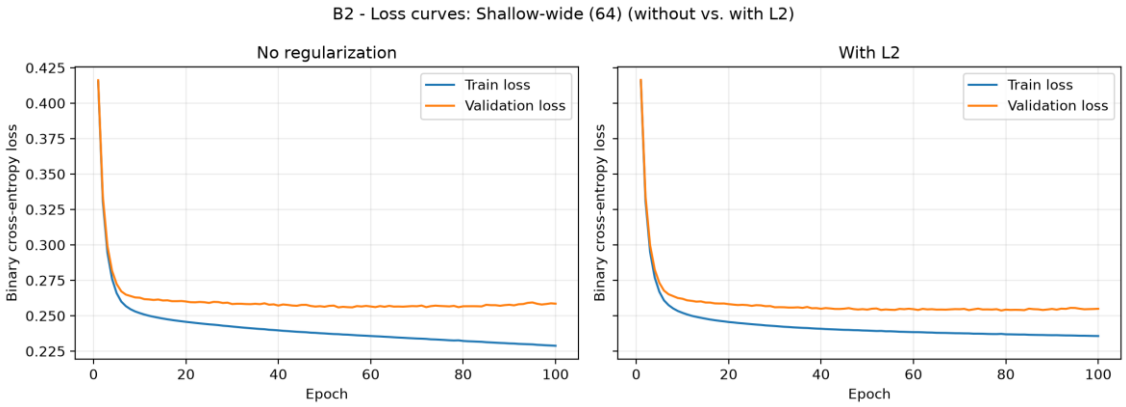


Figure 2 – Loss curves

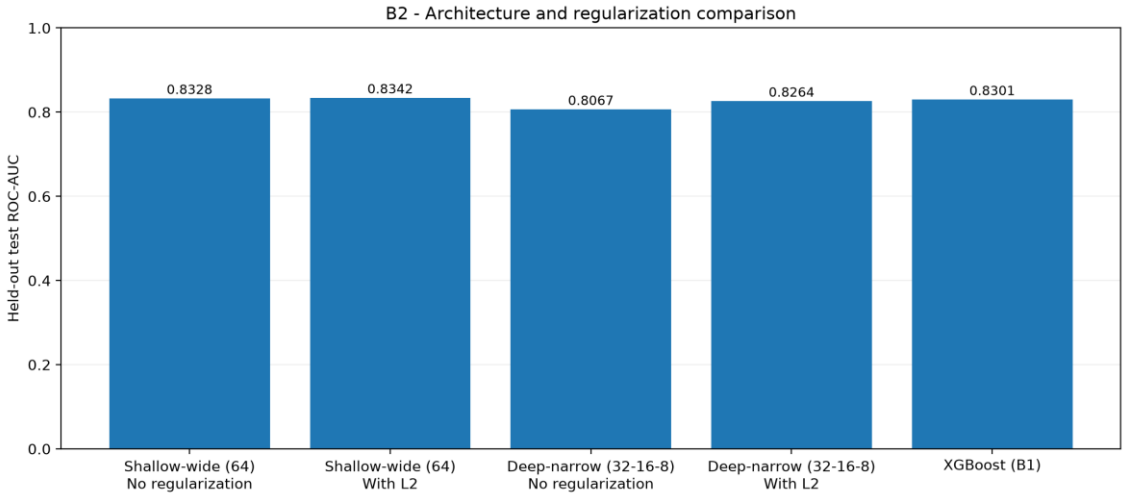


Figure 3 – AUC comparison

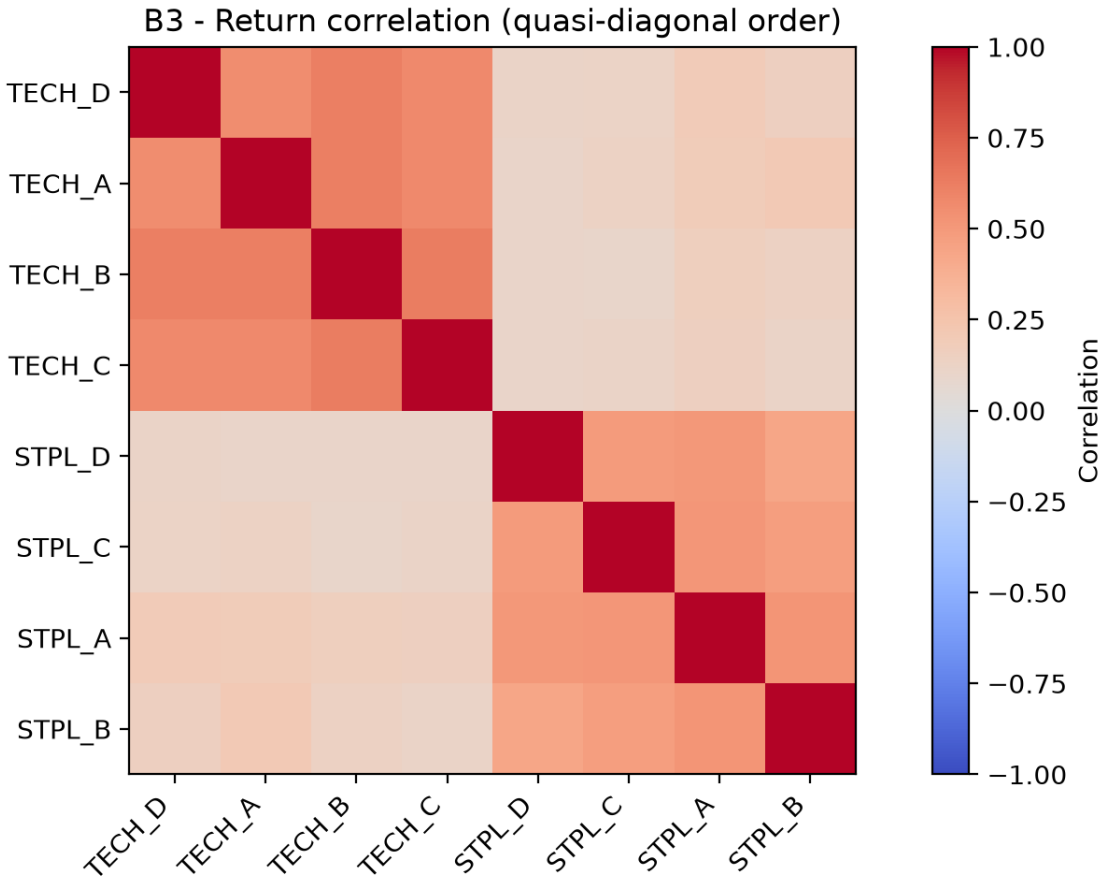


Figure 4 – Correlation Heatmap

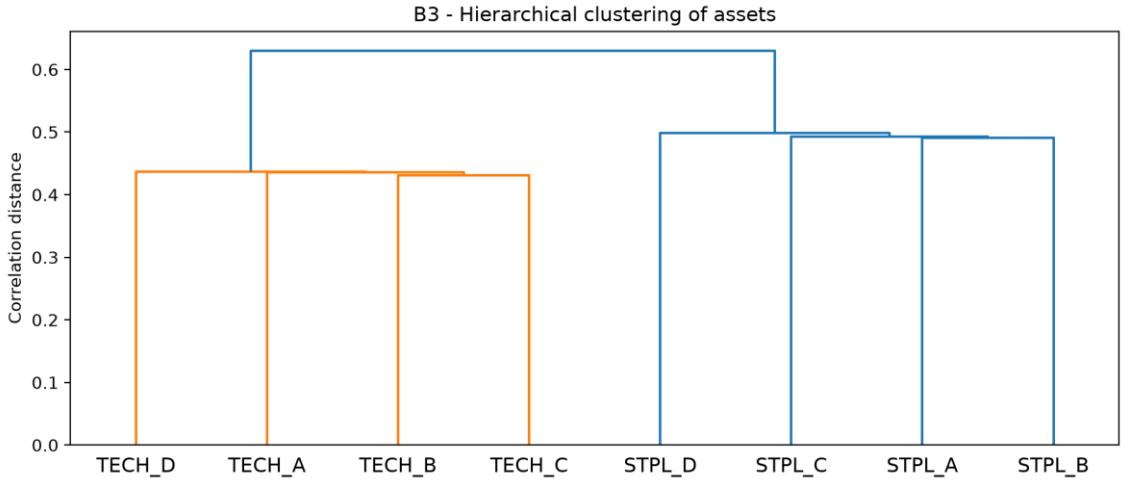


Figure 5 – Hierarchical clustering dendogram

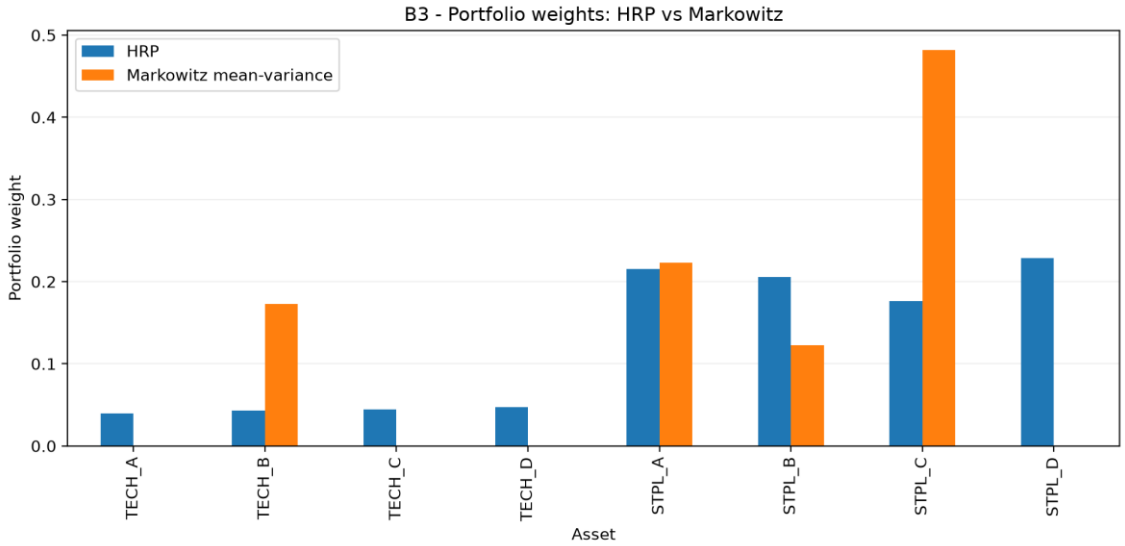


Figure 6 – Weight comparison chart